Dynamic RL-Based LLM Execution Graph Partitioning

Project report: everything done so far, from the first dataset pipeline to the v5 adaptive-scheduling run

Author	Mahaprabhu K
Report date	12 October 2026
Repository	github.com/MahaprabhuK/Dynamic-RL-LLM-Execution-Graph-Partitioning
Working branch	`improve/meaningful-partitioning` (main has the original v1 pipeline)
Model	TinyLlama/TinyLlama-1.1B-Chat-v1.0, float32, CPU only
Hardware	Laptop, 10 physical / 16 logical CPU cores, 16 GB RAM (RTX 3050 present but unused; torch is a CPU build)

Contents

1. Summary
2. Problem and goal
3. Project timeline
4. System design
5. Phase 1 (v1/v2): first pipeline and why its results were noise
6. Phase 2 (v3): real strategies and a corrected methodology
7. Phase 3: contextual-bandit DQN
8. Phase 4: intra-generation re-scheduling (v3 adaptive)
9. Phase 5 (v4): online bandit
10. Phase 6: load sweep
11. Phase 7 (v5): multi-level load and the ramp scenario
12. Methodology and statistics
13. Engineering, testing and reproducibility
14. Limitations and open problems
15. Next steps
16. Appendix: files, commands and data index

1. Summary

The project builds a scheduler that decides, for each LLM request on a single CPU machine, how the model should be executed. TinyLlama-1.1B runs layer by layer through a hand-written decode loop. Its 22 decoder layers are split into stages, and each stage gets its own intra-op thread budget. Three execution strategies exist (`static`, `balanced`, `latency_optimized`). All of them give token-identical output to Hugging Face `model.generate()`, so the choice only affects speed.

The work went through seven phases. The early versions (v1/v2) produced results that turned out to be noise, because the strategies were computationally identical and were measured one after another. v3 fixed both problems. Strategies now differ in thread budgets, and measurements are paired and randomized under controlled background CPU load. A contextual-bandit DQN was then trained on the v3 data. After

that the project moved to its main idea, **re-deciding the strategy every 10 tokens during generation**, plus an online bandit that keeps learning from measured token latency.

> **Main results**
>
> - Which strategy is fastest depends on load. Idle machine: 10 threads (`static`) wins. Around 8 busy workers: 6 threads. At 10-14 busy workers: 3+3 threads (`balanced`). At 12 workers `static` oversubscribes the CPU and is about 18x slower per token than `balanced` (2.14 s vs 0.116 s per token).
> - Re-scheduling during generation pays off when load rises sharply. In the v5 `ramp` scenario (idle, then 8, then 12 workers) the DQN re-deciding every 10 tokens took 54.9 s, against 389.5 s when it decided once per request. That is 85.7% faster (paired 95% CI -430 to -266 s, n=6). In `relief` it was 10.7% faster than deciding once (CI -5.3 to -2.3 s).
> - In v4 the online bandit beat the per-request DQN by 7.8% (`spike`) and 7.0% (`relief`), and corrected a stale offline model.
> - In v5, with harsher load levels, the online bandit was the weakest adaptive policy. It was 57% slower than fixed `balanced` in `spike` and 34% slower than the re-deciding DQN in `ramp`. Section 11 gives the evidence and the likely causes.
> - Output stayed token-identical across all conditions in 100% of contexts in v4 and v5. Decision overhead is 2-5 ms per 200-token request (0.21 s in v3).

2. Problem and goal

LLM inference on a shared machine competes with other processes for CPU. A single fixed execution configuration is only best for one load level. Too many threads under contention cause oversubscription and context switching. Too few threads on an idle machine waste cores. The goal is a scheduler that observes the request and the system state and picks the execution plan (layer partition plus thread budget) that minimizes latency, without changing the model's output.

Most schedulers decide once per request, before the first token. A long response can take tens of seconds, and load can change during that time, so the per-request decision goes stale. The innovation chosen for this project is **intra-generation re-scheduling**. Because every strategy here shares the same weights and the same KV cache, the engine can switch strategy between two tokens at no cost and with identical output.

3. Project timeline

Date	Commit	Work
26 Jul 2026	60ef0b8	Initial project structure
1 Aug 2026	b08ba1d	Dataset pipeline (Alpaca download and preprocessing to `prompts.csv`), TinyLlama inference benchmark
2 Aug 2026	14f631e	LLM inference pipeline, benchmarking, runtime telemetry (CPU/RAM)
15 Sep 2026	34e2043	RL-based execution graph scheduling: DQN agent, environment, execution graph (networkx), static scheduler (current `main`)
10 Oct 2026	ef2f544	Snapshot of the v2 layer-sliced engine and its results, stop tracking `.pyc` files
10 Oct 2026	5291ab2	v3: strategies differ by thread budgets, paired randomized protocol, contextual-bandit framing, tests, config loader, README
10 Oct 2026	3f42bdb	Intra-generation re-scheduling, external-load monitor, dynamic load, v3 dataset, pilot, DQN v3, first adaptive run
11 Oct 2026	6d5a62b	v4: online contextual bandit, non-blocking telemetry, token-triggered load, resume and report-only modes
11 Oct 2026	3c2fc89	Robust bandit (stall rejection, bin hysteresis), load sweep, ramp scenario
11-12 Oct 2026	this commit	v5 run finished overnight (40 contexts x 7 conditions), `--keep-awake` flag, v5 results, this report

4. System design

4.1 Layer-sliced inference engine

`src/llm/inference.py` runs TinyLlama without `model.generate()`. It embeds the prompt, then runs the decoder layers stage by stage with a shared KV cache, applies the final norm and LM head, and samples greedily. Before each stage it sets `torch.set_num_threads()` to that stage's budget. Between stages it records an activation handoff (an in-process `clone()`, timed and sized). The model is loaded lazily and once.

`generate_partitioned_response(prompt, strategy, scheduler=..., decision_interval=N)` accepts a scheduler object. Every N tokens it calls `scheduler.decide()` and switches strategy when asked. All runs in v4/v5 generate exactly 200 tokens (`ignore_eos`), so latencies are comparable across conditions.

4.2 Execution strategies

ID	Strategy	Layer stages	Thread fraction per stage	Threads on this 10-core CPU
0	`static`	0-21	1.0	10
1	`balanced`	0-10, 11-21	0.3, 0.3	3, 3
2	`latency_optimized`	0-21	0.6	6

Strategies are defined in `configs/config.yaml` and loaded by `src/config.py`. `src/llm/partition.py` turns them into stage definitions, and `src/graph/execution_graph.py` builds a networkx execution graph (Input, Tokenizer, LLM, Decoder, Output) annotated with the chosen partition, layer ranges and stage threads.

4.3 Telemetry and load generation

- `ExternalLoadMonitor` measures system CPU minus this process's own CPU, so the scheduler reacts to other processes and not to its own inference threads.

- `BackgroundLoadSampler` (v4) runs that measurement in a daemon thread with smoothing. Reading it never blocks generation. In v3, blocking telemetry cost 0.21 s per request.
- `BackgroundCPULoad` starts N busy-loop worker processes for a whole request. `DynamicCPULoad` changes N over time. `TokenScheduledLoad` (v4) changes N after a given number of generated tokens, so every policy meets the load change at exactly the same point of the response.

4.4 Schedulers and policies

Condition	What it does
`static`, `balanced`, `latency_optimized`	Fixed strategy for the whole request (baselines)
`rule_adaptive`	Threshold rule on external CPU (60% in v5): below it use `static`, above it `balanced`. Re-decided every 10 tokens with hysteresis
`rl_once`	Offline DQN, decides once before the first token
`rl_adaptive`	Same DQN, re-decides every 10 tokens with hysteresis
`bandit_adaptive`	Online contextual bandit (`OnlineBanditPolicy`) that learns from measured seconds per token while it serves

Hysteresis (`patience=2`) means a policy only switches after it proposes the same new strategy in two decisions in a row. This stops flapping on noisy readings, but it also means a switch takes at least 20 tokens after the load changes.

4.5 The online bandit

Each 10-token segment is one bandit round. The context is the external-CPU bin, the arm is the strategy, and the cost is the measured seconds per token of the segment just generated (the first segment, which contains prefill, is skipped). Features:

- Warm start from the offline DQN's Q-values with 2 pseudo-observations per arm.
- Discounted estimates (discount 0.98 per round in a bin) so stale data fades when the machine drifts.
- UCB-style exploration bonus (0.08) so arms not tried recently are re-checked, at the price of one segment.
- Stall rejection (v4.1): a segment more than 2x slower than the best estimate in its bin is ignored, unless 5 arrive in a row, in which case the bin is reset and relearned.
- Bin hysteresis: the bin only changes within a request when the reading moves more than `bin_margin` points past an edge (v5: edges 51% and 63%, margin 4).
- One instance keeps learning across all requests. Its state is saved to `adaptive_bandit_state_*.json` so interrupted runs can resume.

4.6 Offline RL component

The DQN (`src/rl/dqn_agent.py`) is an MLP with two hidden layers of 64 units, input `[prompt_length, input_tokens, cpu%, ram%]` and one Q-value per strategy. Since requests are independent, it is trained as a contextual bandit with `gamma=0`. The logged data contains the outcome of every strategy in every context, so all three rewards are learned per context (full feedback). The reward is negative seconds per generated token. CPU and RAM penalties exist in `src/rl/reward.py` but are off by default, because post-run snapshots were too noisy. Features are built by one shared function (`src/rl/features.py`) for training and serving.

5. Phase 1 (v1/v2): first pipeline and why its results were noise

The `main` branch has the first full pipeline: Alpaca prompts, TinyLlama inference, CPU/RAM telemetry, a DQN scheduler and evaluation scripts. The v2 work then added the layer-sliced engine. Results from that stage:

Run	static	balanced	latency_optimized	Note
v1 fixed strategies, mean latency (20 prompts)	14.83 s	25.14 s	22.60 s	Sequential runs
v2 fixed strategies, mean latency (20 prompts)	24.08 s	22.73 s	22.36 s	Sequential runs
v2 RL policy evaluation	25.12 s	12.45 s	19.74 s	RL picked `latency_optimized` 20/20 times, 19.37 s

The rankings flip between runs. Review of the code showed why: in v1/v2 the three strategies did the same computation with the same threads, only grouped differently. All runs of one strategy were also done before the next one started, so system drift (other apps, thermal state, memory pressure) was confounded with the strategy. The differences were noise. These files are kept as `*_v2.csv` for reference and are not used for conclusions.

6. Phase 2 (v3): real strategies and a corrected methodology

Commit 5291ab2 changed the strategies so they execute differently (per-stage thread budgets, still fp32 and token-identical). It also changed the measurement protocol. Strategies are now run back to back in random order within the same context (prompt, repeat, load level), after warm-up, under a controlled number of background busy workers. The state is measured before the decision, real generated token counts are used, and results come with bootstrap 95% CIs and paired differences.

6.1 Pilot (3 prompts, 0/4/8 workers)

Workers	static (10 thr)	balanced (3+3)	latency_opt (6)	Paired result vs static
0	6.70 s	8.26 s	8.17 s	balanced +28.8% slower (significant)
4	20.98 s	18.19 s	17.27 s	latency_opt -14.2% (significant)
8	72.63 s	41.95 s	28.13 s	latency_opt -61.2%, balanced -42.0% (both significant)

This was the first evidence that the strategies have real, load-dependent winners.

6.2 v3 dataset (10 prompts x 5 load levels x 3 strategies = 150 runs, 50 tokens each)

Workers	static	balanced	latency_opt	balanced vs static (paired)	latency_opt vs static (paired)
0	8.70 s	11.27 s	9.35 s	+30.1% (sig.)	+11.8% (n.s.)
2	8.42 s	8.40 s	8.20 s	+6.1% (n.s.)	+0.6% (n.s.)
4	19.42 s	12.81 s	16.62 s	-27.3% (sig.)	-16.2% (sig.)
6	24.71 s	15.94 s	22.40 s	-26.4% (sig.)	-17.7% (n.s.)
8	44.27 s	22.87 s	45.32 s	-34.9% (sig.)	+65.8% (n.s.)

Conclusion from v3: `static` wins on an idle machine, `balanced` wins from 4 busy workers upward. The threshold baseline was set to static/balanced from these numbers.

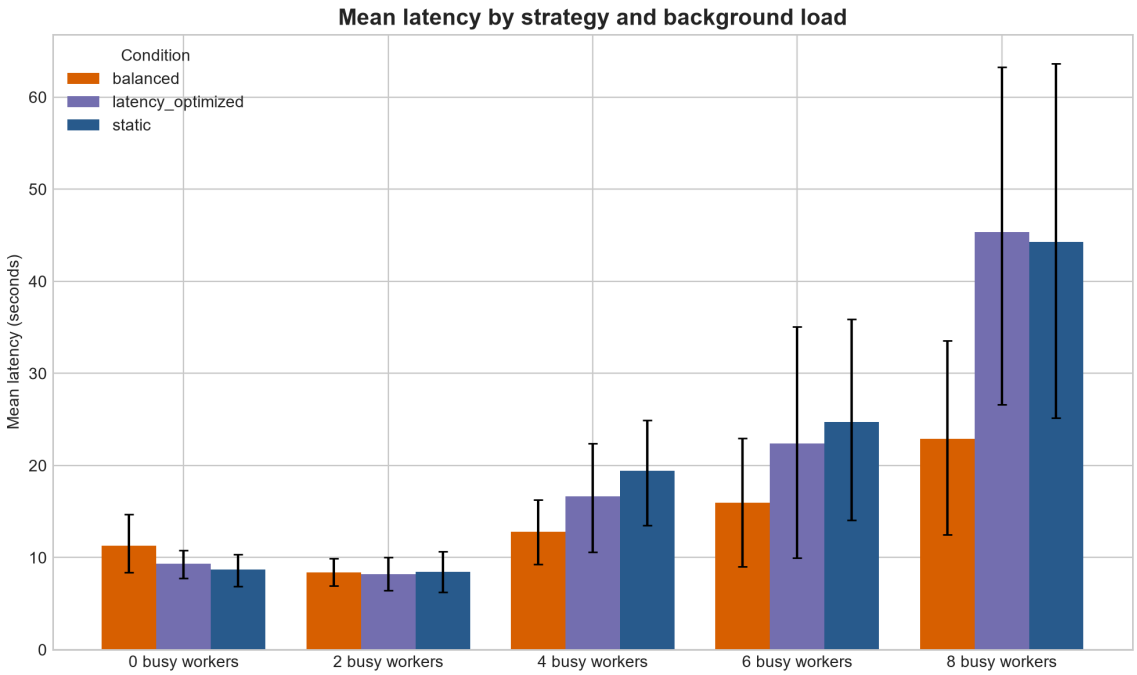


Figure 1. v3 mean latency per strategy and background load (bootstrap 95% CIs).

7. Phase 3: contextual-bandit DQN

The DQN was trained for 200 episodes on the v3 data with full feedback, with prompts split into train and held-out validation sets. On the 15 held-out contexts it had zero regret in 6 of them and a mean regret of 0.049 s per token against the oracle (best strategy in hindsight). It learned the main pattern in the data: mostly `static` at 0-2 workers and `balanced` at 4-8 workers. The model is saved as `data/processed/dqn_model_v3.pth` and is the offline policy used in all later runs.

8. Phase 4: intra-generation re-scheduling (v3 adaptive)

Commit 3f42bdb added re-scheduling during generation and `evaluate_adaptive.py`. Four scenarios: `steady_idle`, `steady_heavy`, `spike` (load arrives mid-request) and `relief` (load leaves mid-request). Six conditions were compared on 60-token requests with time-triggered load, 3 prompts per scenario.

Result: no benefit. `rl_adaptive` was not significantly different from `rl_once` in any scenario, and was 35% slower than fixed `latency_optimized` in `spike`. Three causes were found: 60-token requests were too short for a mid-request change to matter, the time-triggered load did not hit every policy at the same token, and blocking telemetry cost 0.21 s per request. RAM pressure had also changed since the v3 data was collected, so the offline DQN was already stale. These findings drove the v4 design.

9. Phase 5 (v4): online bandit

v4 setup: 10 prompts x 4 scenarios = 40 contexts, 7 conditions, 200 tokens per request, decision every 10 tokens, load triggered by token count, non-blocking telemetry. Two contexts with memory-paging stalls (a run more than 2x the scenario median) were excluded. Raw data is kept.

Scenario	balanced	bandit	latency_opt	rl_adaptive	rl_once	rule_adaptive	static
relief	20.98	19.09	17.96	19.68	20.61	19.62	20.16
spike	24.75	22.69	22.70	24.04	24.63	24.86	24.17
steady_heavy	25.21	24.20	23.27	25.13	25.16	25.25	27.33
steady_idle	19.60	17.29	17.18	17.52	17.77	17.46	17.27

Mean latency in seconds for a 200-token response. v4 heavy load was 8 workers.

- The bandit was significantly faster than the per-request DQN when load changed mid-request: -7.8% in `spike`, -7.0% in `relief`. It was also significantly faster than `rl_adaptive` in `spike` and `steady_heavy`, and than the threshold rule in three scenarios.
- It corrected a stale offline model. The v3 data said `balanced` wins under load. At v4 time the 6-thread `latency_optimized` won everywhere. The DQN kept choosing `balanced`; the bandit moved to `latency_optimized`.
- Regret against the best fixed strategy in hindsight: 0.3-1.2 s per request for the bandit, 0.9-2.1 s for the DQN. Bandit regret fell from 0.94 s to 0.66 s between the first and second half of the run, so it was learning.
- Limitation: one fixed strategy happened to be near-optimal everywhere, so the bandit only tied it in `spike` and `steady_idle` and was 4-6% slower in `relief` and `steady_heavy`. That is the cost of exploration (about 3 switches per request).
- Decision overhead fell from 0.21 s (v3) to 0.002 s per request.

v4 diagnostics also showed two bandit problems, both fixed in commit 3c2fc89. Paging stalls had poisoned its estimates, which led to stall rejection and bin reset. Readings near a bin edge caused bin flapping, 53 of 114 switches, which led to bin hysteresis. Forgetting was also slowed (discount 0.98) and the exploration bonus reduced (0.08).

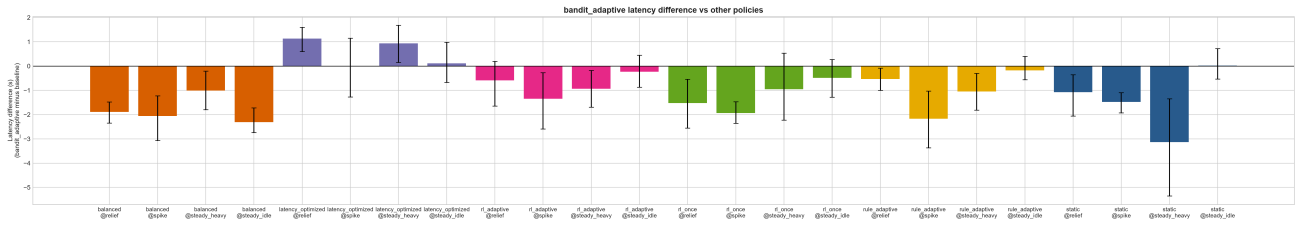


Figure 2. v4 paired differences of the online bandit against every other condition (negative = bandit faster).

10. Phase 6: load sweep

Because v4 had only one winning strategy, `pilot_load_sweep.py` measured decode seconds per token for every strategy at 0-14 busy workers (2 repeats) to find load levels where the winner changes.

Workers	static (10 thr)	latency_opt (6)	balanced (3+3)	Best
0	0.0790	0.0793	0.0879	static
2	0.0799	0.0892	0.0899	static
4	0.0827	0.0917	0.0998	static
6	0.0930	0.0938	0.1053	static
8	0.1099	0.0965	0.1072	latency_optimized
10	0.1223	0.1135	0.1087	balanced
12	2.1352	0.1510	0.1157	balanced (static 18x slower)
14	3.0548	0.8520	0.2155	balanced

Decode seconds per token, mean of 2 repeats (`data/processed/load_sweep_v4.csv`). External CPU readings: 7-49% at 0-6 workers, 54-60% at 8, 66-89% at 10-14.

The sweep shows three regimes and a cliff. Up to 6 workers all strategies are within about 15% of each other. At 12 workers `static` collapses because 10 inference threads plus 12 busy processes oversubscribe 16 logical cores. These regimes set the v5 bandit bin edges (51%, 63%) and the rule threshold (60%).

11. Phase 7 (v5): multi-level load and the ramp scenario

v5 setup: 8 prompts (prompt IDs 21-28, unseen in training) x 5 scenarios = 40 contexts, 7 conditions, 280 requests of 200 tokens each. Heavy phases use 10 workers, load changes at token 60. The new `ramp` scenario goes idle, then 8 workers from token 60, then 12 from token 140, so no single fixed strategy is best for the whole response. The run was paused on 11 October with 5 contexts done. One paging-corrupted context was removed (a backup is in `data/processed/v5_backup`). A Windows scheduled task then resumed it at midnight with `--resume --keep-awake`, and it finished at 04:56 on 12 October. Four contexts with paging stalls (IDs 14, 19, 36, 39) are excluded, leaving 36 (6-8 per scenario). Token output was identical across all conditions in 100% of contexts. Mean decision overhead: 0.005 s per request.

11.1 Mean latency (seconds, 200 tokens)

Scenario	balanced	bandit	latency_opt	rl_adaptive	rl_once	rule_adaptive	static
ramp (n=6)	60.0	76.4	69.5	**54.9**	389.5	55.5	412.6
relief (n=7)	33.8	33.6	33.5	30.7	34.4	**30.2**	77.3
spike (n=8)	**37.7**	59.2	45.3	59.5	86.0	59.4	133.2
steady_heavy (n=7)	34.1	42.9	45.7	38.7	34.3	**33.8**	91.5
steady_idle (n=8)	29.7	23.7	22.8	22.8	**22.4**	22.5	22.7

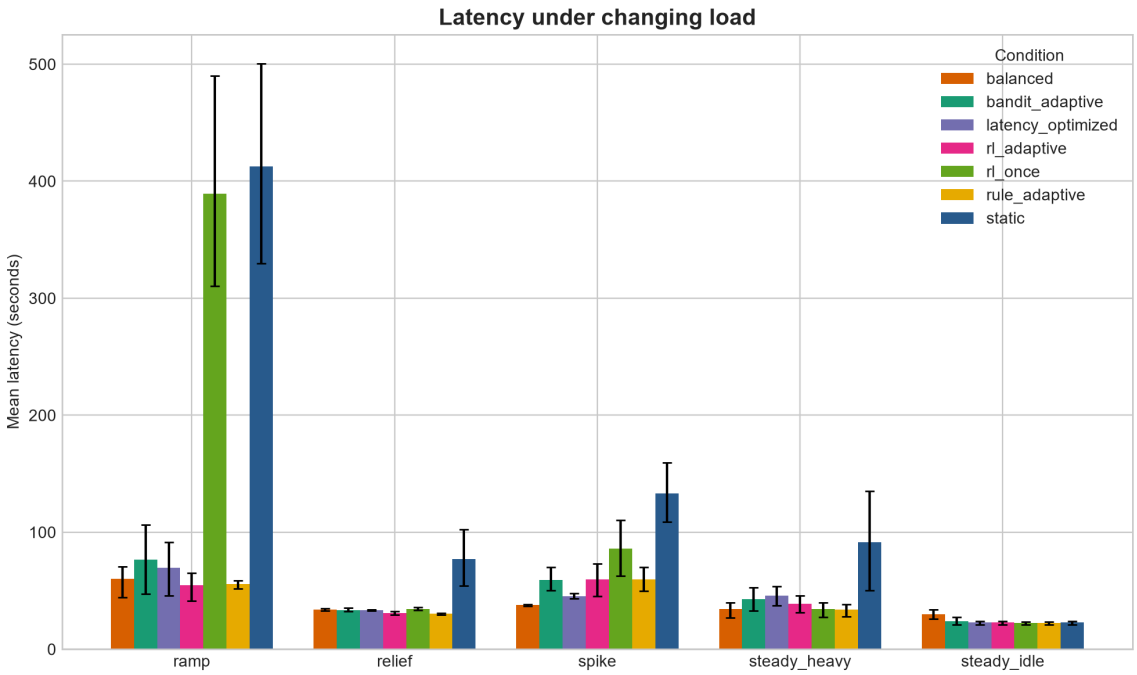


Figure 3. v5 mean latency by scenario and condition with bootstrap 95% CIs.

11.2 Re-deciding vs deciding once

Scenario	rl_adaptive vs rl_once	95% CI (s)	Verdict
ramp	-334.6 s (-85.7%)	[-429.7, -266.2]	adaptive much faster
relief	-3.7 s (-10.7%)	[-5.3, -2.3]	adaptive faster
spike	-26.5 s (-12.8%)	[-55.3, 3.3]	not significant
steady_heavy	+4.4 s (+17.0%)	[-0.3, 12.8]	not significant
steady_idle	+0.4 s (+1.9%)	[0.1, 0.8]	once slightly faster

This is the clearest result of the project so far. In `ramp` the DQN sees an idle machine at the start and picks `static`. Deciding once, it keeps 10 threads when 12 workers arrive at token 140 and hits the oversubscription cliff (389.5 s). Re-deciding every 10 tokens it moves away from `static` and finishes in 54.9 s, 7.7% faster than the best fixed strategy (`balanced`, paired CI -7.1 to -2.7 s). In `relief`, both `rl_adaptive` and `rule_adaptive` beat every fixed strategy by about 9-10% (significant against `balanced` and `latency_optimized`), because they move to `static` once the load leaves.

11.3 The spike scenario: switching too late

In `spike` all three adaptive policies (rule, DQN, bandit) were about 58% slower than fixed `balanced` (rule: +21.8 s, CI 11.8 to 32.1). The per-token timeline (Figure 4) shows why. The policies run `static` while the machine is idle. When 10 workers arrive at token 60, `static` slows to about 2.5 s per token, and the policies need 10-15 more tokens to switch: one decision interval (10 tokens) plus hysteresis (2 matching decisions), plus telemetry smoothing. Those few tokens at 2.5 s cost more than everything else. Fixed `balanced` pays about 10% on the idle part but never hits the cliff.

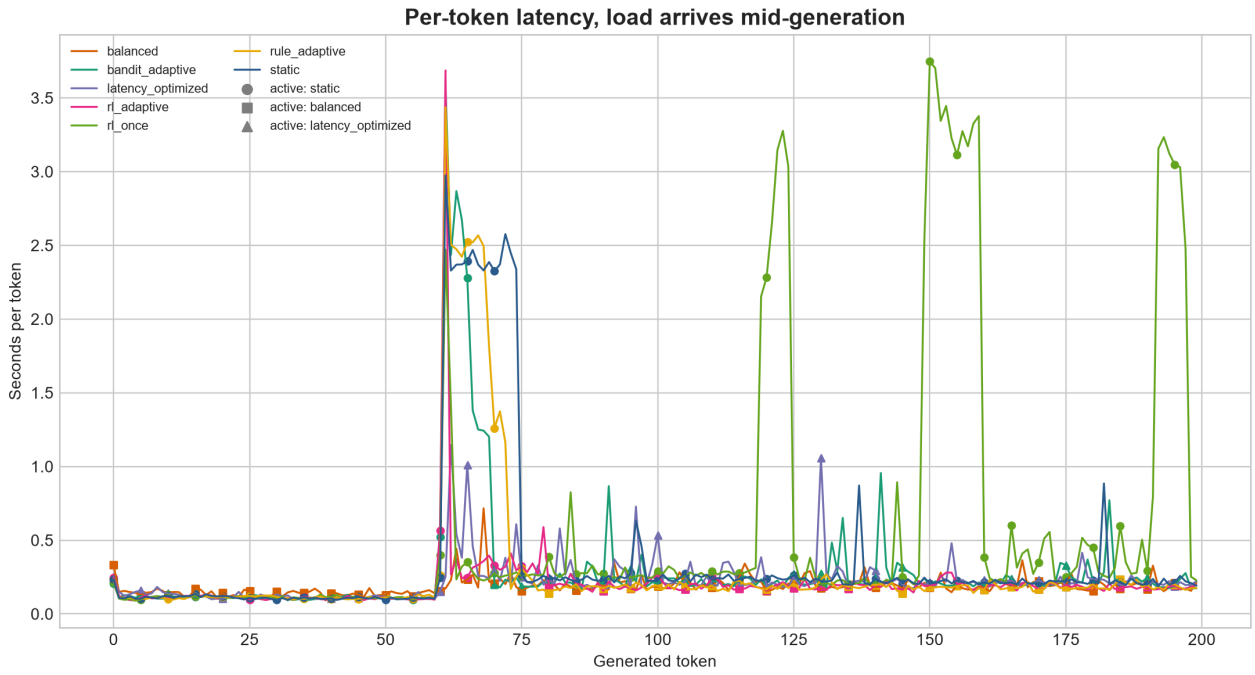


Figure 4. v5 per-token latency for one spike context. Load arrives at token 60; markers show the active strategy at each decision.

11.4 The online bandit in v5

Scenario	bandit vs best fixed	bandit vs rl_adaptive	bandit vs rule_adaptive
ramp	+22.4% vs balanced (n.s.)	+33.7% (sig.)	+33.8% (n.s.)
relief	+0.3% vs latency_opt (n.s.)	+9.9% (sig.)	+11.2% (sig.)
spike	+57.1% vs balanced (sig.)	+15.7% (n.s.)	+4.3% (n.s.)
steady_heavy	+24.4% vs balanced (sig.)	+12.2% (n.s.)	+24.6% (sig.)
steady_idle	+4.1% vs latency_opt (n.s.)	+3.9% (n.s.)	+5.3% (n.s.)

The v4 result did not carry over. The bandit's logged history explains part of it. In the high-load bin (above 63% external CPU) it accepted 104 segments of `balanced` with a median of 0.132 s per token, and 267 segments of `latency_optimized` with a median of 0.234 s per token, so it chose the slower arm about 2.5 times as often. In `spike` it never chose `balanced` at all (101 decisions on `latency_optimized`, 59 on `static`). Likely contributors, not yet confirmed:

- The DQN warm start favours `latency_optimized`/`static` in bins it was not trained on (the v3 data stopped at 8 workers, about 75% CPU).
- Per-segment cost includes the time spent before a switch takes effect, so the arm active right after a load change gets blamed for the previous arm's slow tokens, or the reverse.
- The stall rule (reject segments more than 2x slower than the best estimate) can throw away genuine evidence when a load level really does make one arm 2x slower.
- The bandit keeps exploring `static` in the low bin (324 segments) and pays for it when load arrives within that request.

Learning still happened. Mean regret against the best fixed strategy fell from 12.88 s in the first half of the run to 7.85 s in the second half. It stayed above `rl_adaptive` (6.30 s to 4.22 s) in both halves.

> Honest reading of v5: re-scheduling during generation is clearly worth it when load rises past the oversubscription cliff (ramp: -86% against deciding once). The simple threshold rule and the re-deciding DQN got most of that gain. The online bandit, which was the best policy in v4, was the worst adaptive policy in v5, and needs to be fixed before it can be claimed as a contribution.

12. Methodology and statistics

- Paired, randomized measurements: all conditions run back to back on the same prompt under the same load scenario, in shuffled order, after a warm-up generation. This removes the run-order confound of v1/v2.
- Pre-decision state: `[prompt_length, input_tokens, cpu%, ram%]` is measured before the request, identically in training and serving.
- Fixed-length requests: v4/v5 generate exactly 200 tokens, so latency is comparable.
- Load change at a token index, not a time, so every policy faces it at the same point of the response.
- Statistics: bootstrap 95% CIs on means and on paired per-context differences. A difference is called significant only if its CI excludes zero. Regret is measured against the best fixed strategy in hindsight per context.
- Stall exclusion: a context is excluded if any run is more than 2x the median of the same condition in the same scenario. Comparing within a condition keeps policies that are slow every time (for example `static` under oversubscription). Raw data is never deleted.
- Output equivalence is checked on token IDs for every context.
- Handoff timings are in-process `clone()` copies, a lower bound on a real inter-device transfer.

13. Engineering, testing and reproducibility

- Tests: `tests/test_pipeline.py` (8 tests) and `tests/test_adaptive.py` (15 tests) run without the model. All 25 collected test cases passed on 12 October 2026 (55.7 s). `tests/test_functional_equivalence.py` loads the model and checks that every strategy reproduces `model.generate()` token for token.
- `evaluate_adaptive.py` supports `--resume` (skip finished contexts and restore bandit state), `--report-only` (re-analyse saved CSVs), configurable scenarios (`--heavy`, `--switch-token`, `--ramp`), bandit bins (`--bandit-edges`, `--bin-margin`) and, in this commit, `--keep-awake`, which stops Windows from sleeping during long runs through `SetThreadExecutionState`.
- `run_overnight.cmd` resumes the v5 run unattended with `HF_HUB_OFFLINE=1` and appends to a log.
- Every data version is written under its own tag (v2, pilot, v3, v4, v5). Nothing older is overwritten.
- Environment quirks: Hugging Face downloads needed a TLS workaround (truststore); scripts run with `HF_HUB_OFFLINE=1`. The fp32 model needs about 4.4 GB. With 16 GB RAM and RAM usage at 92-97% during runs, paging stalls are the main source of outliers.

14. Limitations and open problems

1. Small samples: 6-8 contexts per scenario in v5, 3 in the v3 adaptive run. Several CIs are wide (ramp bandit vs balanced: -2.3 to 37.8 s).
2. One machine, one model, CPU only. The thread-budget regimes are specific to this 10-core / 16-thread laptop and will move on other hardware.
3. Memory pressure: RAM at about 95% causes paging stalls, which required excluding 2 (v4) and 4 (v5) contexts and removing one corrupted v5 context by hand.

4. Synthetic load: busy-loop worker processes are a simple model of contention. Real co-located workloads also compete for memory bandwidth and cache.
5. Reaction delay: with a 10-token interval and patience 2, policies switch too late when load arrives suddenly, which cost about 58% in `spike`.
6. Online bandit underperforms in v5 (Section 11.4).
7. The offline DQN goes stale when machine conditions change (seen between v3 and v4). It was trained on loads up to 8 workers only.
8. The "partitioning" is within one device. Layer stages run sequentially, so stage splits only change thread budgets, not parallelism. A CPU/GPU split (RTX 3050) has not been tried.
9. The execution graph in `src/graph` is descriptive (5 coarse nodes plus partition metadata). The scheduler does not search over graph partitions.

15. Next steps

1. Fix the bandit: attribute cost only to tokens generated after a switch takes effect, retest the stall rule at high load, cap the influence of the DQN prior in bins outside its training range, and add a safety rule that never explores `static` above the cliff.
2. Faster reaction to load spikes: an emergency switch when per-token latency jumps (for example 3x the recent median), without waiting for the next interval or for hysteresis. This targets the `spike` loss directly.
3. Retrain the DQN on the v4/v5 logs and the load sweep, so it covers 10-14 workers.
4. More contexts per scenario (at least 15) once memory pressure is reduced, for example with bfloat16 (halves RAM, outputs change slightly) or by closing other applications.
5. Extend strategies to a CPU/GPU split on the RTX 3050 (needs a CUDA build of torch), which would make the partitions real device partitions.

16. Appendix: files, commands and data index

16.1 Repository layout

```
configs/config.yaml            model + strategy definitions
src/config.py                  config loader
src/data/                      Alpaca download, preprocessing, prompt loading
src/llm/                       model loading, partition strategies, layer-sliced inference
src/graph/execution_graph.py   networkx execution graph annotated with partitions
src/telemetry/                 CPU/RAM metrics, external-load monitor, load generators
src/rl/                        features, reward, offline bandit environment, DQN agent, training
src/scheduler/                 static, RL and adaptive (rule / DQN / online bandit) policies
src/benchmark/                 bootstrap statistics and plots
evaluate_fixed_strategies.py   paired fixed-strategy dataset (v3, pilot)
evaluate_rl_policy.py          online evaluation of the trained DQN
evaluate_adaptive.py           intra-generation re-scheduling under changing load (v3-v5)
pilot_load_sweep.py            per-strategy decode speed at 0-14 busy workers
run_overnight.cmd              unattended resume of the v5 run
tests/                         pipeline, adaptive and model-equivalence tests
docs/report/                   this report (HTML source and PDF)
```

16.2 Reproducing the results

```
python -m src.data.download_dataset
python -m src.data.preprocess_dataset
python -m pytest tests/test_functional_equivalence.py -v
python evaluate_fixed_strategies.py --num-prompts 10 --repeats 1 --loads 0,2,4,6,8 --max-new-tokens 50
python -m src.rl.train --csv data/processed/fixed_strategy_metrics_v3.csv --tag v3
python pilot_load_sweep.py
python evaluate_adaptive.py --tag v4 --num-prompts 10 --prompt-offset 20 --max-new-tokens 200
python evaluate_adaptive.py --tag v5 --num-prompts 8 --prompt-offset 20 --heavy 10 --switch-token 60 \
    --ramp "60:8,140:12" --threshold 60 --bandit-edges "51,63" --bin-margin 4 --resume --keep-awake
python evaluate_adaptive.py --tag v5 --report-only
python -m pytest tests/test_pipeline.py tests/test_adaptive.py
```

16.3 Data index (data/processed)

Files	Contents
`prompts.csv`, `inference_results.csv`, `runtime_metrics.csv`	v1 prompts, inference and telemetry
`fixed_strategy_*_v2.csv`, `rl_*_v2.csv`, `dqn_model_v2.pth`	v1/v2 results (noise, reference only)
`fixed_strategy_*_pilot.csv`	Pilot: 3 prompts x 0/4/8 workers
`fixed_strategy_*_v3.csv`	v3 paired dataset, stage-level metrics, paired diffs
`rl_training_history_v3.csv`, `rl_offline_evaluation_v3.csv`, `dqn_model_v3.pth`	DQN training and held-out evaluation
`adaptive_*_v3.csv`	First re-scheduling run (60 tokens)
`adaptive_*_v4.csv`, `adaptive_report_v4.txt`	v4 runs, decisions, per-token timelines, bandit history, summary, paired diffs
`load_sweep_v4.csv`	Seconds per token per strategy at 0-14 workers
`adaptive_*_v5.csv`, `adaptive_report_v5.txt`, `adaptive_bandit_state_v5.json`	v5 runs with the ramp scenario

Plots for every version are in `results/` (latency by load or scenario, paired differences, spike and relief timelines).